Project Milestone Guide for Students

A generalized, step-by-step framework to plan, build and submit your web project

Project Name: _____

Team Members: _____

Mentor / Trainer: _____

---

1. How to Use This Guide

Every real-world software project moves through the same stages: **understand the problem, design the solution, build it, test it, deploy it, and present it.** This guide breaks that journey into **6 milestones**.

- This document gives you a **basic idea** of what each milestone means and what you are expected to produce.
- **You must create your own milestone plan based on your project.** The examples here are only a starting point, not a copy-paste answer.
- For each milestone, write your project name and replace every generic point with something specific to *your* project.
- Each milestone ends with a **deliverable checklist**. Do not move to the next milestone until the checklist is complete.

Example of making it specific to your project:

Generic (this guide)	Specific (your work)
User Management	"Admin can add, edit and block students in the Library System"
Core Features	"Search books, issue book, return book, calculate fine"
Payment Integration	"Razorpay test-mode integration for fine payment"

---

2. Milestone Overview

#	Milestone	Main Question It Answers	Key Output
1	Requirement	*What* are we building and for *whom*?	SRS document

#	Milestone	Main Question It Answers	Key Output
2	Design	*How* will it be structured?	Architecture, ER diagram, API design, wireframes, **Cost of the project ,if it goes to clients/customers.**
3	Development	Let's build it	Working application code
4	Testing	Does it work correctly?	Test cases, bug reports, fixed code
5	Deployment	Can anyone use it online?	Live application
6	Final Submission	Can we present and document it?	Report, PPT, demo, repo, live URL

Suggested time split (adjust with your mentor): Requirement 10% | Design 15% | Development 40% | Testing 15% | Deployment 10% | Final Submission 10%

---

3. Milestone 1 — Requirement

Goal: Clearly understand the problem and decide exactly what your project will and will not do, *before* writing any code.

Terminology

SRS (Software Requirements Specification) A formal document that describes everything the software should do and how it should behave. It is the agreement between the developer and the client (or mentor). It usually contains: purpose, scope, user types, functional requirements, non-functional requirements, and constraints.

Use Cases Descriptions of how a user interacts with the system to achieve a goal. Each use case has an **actor** (who), a **goal** (what), and **steps** (how). *Example:* Actor: Student → Goal: Borrow a book → Steps: Login → Search book → Click "Issue" → Receive confirmation. Use cases are often drawn as a **Use Case Diagram** (stick figures and ovals).

Functional Requirements What the system **must do** — the features and behaviours. Usually written as "The system shall...". *Example:* "The system shall allow users to register using email and password."

Non-functional Requirements How **well** the system must work — quality attributes rather than features. Common categories:

- **Performance** — pages load within 3 seconds

- **Security** — passwords are encrypted, role-based access
- **Usability** — easy to use on mobile and desktop
- **Scalability** — can handle growth in users
- **Reliability / Availability** — works 99% of the time
- **Maintainability** — clean, documented, modular code

What to Do for Your Project

1. Write the problem statement (2–3 lines).
2. Identify user roles (e.g., Admin, User, Guest).
3. List 5–10 use cases.
4. Write functional requirements per module.
5. Write at least 5 non-functional requirements.
6. Define scope: what is **included** and what is **out of scope**.

Deliverables Checklist

- ☐Problem statement
- ☐SRS document
- ☐Use case list / diagram
- ☐Functional requirements list
- ☐non-functional requirements list

---

4. Milestone 2 — Design

Goal: Plan the blueprint of your system so that development becomes organized and fast.

Terminology

Architecture The overall structure of the system: which major components exist and how they communicate. A common web architecture is **3-tier**:

1. **Presentation layer** — Frontend (what the user sees)
2. **Application layer** — Backend (business logic and APIs)
3. **Data layer** — Database (stored data)

You should also decide your **tech stack** (e.g., React + Spring Boot + MySQL) and draw an architecture diagram showing how the parts connect.

ER Diagram (Entity-Relationship Diagram) A visual map of the data in your system:

- **Entity** — a thing you store data about (User, Book, Order)
- **Attribute** — a property of an entity (name, email, price)
- **Relationship** — how entities connect (One User *places* Many Orders)
- **Cardinality** — one-to-one, one-to-many, many-to-many

Database Design Turning the ER diagram into real tables. Includes:

- **Tables and columns** with data types
- **Primary Key** — a unique identifier for each row
- **Foreign Key** — a column linking one table to another
- **Constraints** — rules like NOT NULL, UNIQUE
- **Normalization** — organizing tables to avoid duplicate data

API Design Planning the communication points between frontend and backend *before* building them. For each API, define:

- **Endpoint (URL)** — e.g., `/api/users`
- **HTTP method** — GET (read), POST (create), PUT/PATCH (update), DELETE (remove)
- **Request** — data sent (body, parameters)
- **Response** — data returned and status codes (200 OK, 201 Created, 400 Bad Request, 401 Unauthorized, 404 Not Found, 500 Server Error)

Example: `POST /api/auth/login` → request: email, password → response: token + user details.

UI Wireframes Simple, low-detail sketches of each screen showing layout and navigation — no colors or final styling. They can be hand-drawn or made in tools like Figma, Balsamiq or Canva. A **mockup** is a more polished, colored version of a wireframe.

What to Do for Your Project

1. Choose your tech stack and justify it.
2. Draw the architecture diagram.
3. Draw the ER diagram.
4. Design tables with keys.
5. List all APIs in a table.
6. Create wireframes for every major screen (login, dashboard, main feature pages).

Deliverables Checklist

- ☐Architecture diagram
- ☐ER diagram
- ☐Database schema (tables, keys)
- ☐API list (endpoint, method, request, response)
- ☐Wireframes of all key screens

---

5. Milestone 3 — Development

Goal: Convert your design into a working application, built in small, testable pieces.

Terminology

Authentication Verifying **who the user is** — register, login, logout, password reset. Common approaches: session-based login, **JWT (JSON Web Token)**, OAuth / social login. Passwords must always be stored **hashed** (e.g., BCrypt), never as plain text.

Authorization Deciding **what a logged-in user is allowed to do**. Usually implemented as **role-based access control (RBAC)** — e.g., only Admin can delete users.

User Management Features to create, view, update and deactivate user accounts, manage profiles, and assign roles. It is typically built right after authentication.

Core Features The main functions that solve your project's problem — the reason your project exists. This is the heart of your project and should get the most time. Includes **CRUD operations** (Create, Read, Update, Delete) on your main entities, plus project-specific logic such as search, filters, bookings, calculations, or dashboards.

Frontend The part users see and interact with, built using HTML, CSS, JavaScript or frameworks like React, Angular or Vue. Includes pages, forms, validation, navigation, and a **responsive design** that works on mobile and desktop.

Backend The server-side part that handles business logic, processes requests, talks to the database and exposes APIs. Examples: Spring Boot (Java), Node.js/Express, Django, Flask.

Integration Connecting the parts together and with outside services:

- **Frontend ↔ Backend** — calling your APIs from the UI
- **Backend ↔ Database** — saving and fetching data
- **Third-party services** — payment gateway, email/SMS, maps, cloud storage, AI APIs

What to Do for Your Project

1. Set up the project structure and a **GitHub repository** (commit regularly with meaningful messages).(**do not do until I(Mentor) tell)**
2. Build authentication first, then user management.
3. Build core features one module at a time (backend API → frontend screen → connect them).
4. Integrate any third-party service you planned.
5. Keep checking the work against your SRS.

Deliverables Checklist

- ☐Login / registration working
- ☐User and role management working
- ☐All core features working end to end
- ☐Frontend connected to backend
- ☐Code pushed to GitHub with regular commits

---

6. Milestone 4 — Testing

Goal: Find and fix problems before real users do.

Terminology

Unit Testing Testing the smallest pieces of code (a single function or method) in isolation. Tools: JUnit/Mockito (Java), Jest (JavaScript), PyTest (Python).

API Testing Checking that each API returns the correct data and status codes for valid *and* invalid requests. Tools: Postman, Swagger UI, Thunder Client.

Integration Testing Checking that different modules work correctly **together** — e.g., frontend → backend → database flow for placing an order.

UI Testing Checking the user interface: buttons, forms, validation messages, navigation, and responsiveness on different screens and browsers. It can be manual or automated (Selenium, Cypress).

Bug Fixing The cycle of recording a defect, fixing it, and re-testing. Every bug report should have: description, steps to reproduce, expected vs actual result, severity, and status (Open / Fixed / Verified). Re-testing after fixes is called **regression testing**.

Test Case A written scenario with input, steps and expected result. *Example:* Login with wrong password → expected: error message "Invalid credentials".

What to Do for Your Project

1. Write test cases for every major feature (include valid, invalid and edge cases).
2. Run unit tests on important logic.
3. Test every API in Postman and keep screenshots.
4. Test full user journeys.
5. Maintain a bug log and fix all high-priority bugs.

Deliverables Checklist

- ☐Test case document
- ☐Unit test results
- ☐API test screenshots / collection
- ☐Integration and UI test results
- ☐Bug log with status

---

7. Milestone 5 — Deployment

Goal: Put your project on the internet so anyone can use it.

Terminology

Production Build An optimized, final version of your code prepared for real users (minified, no debug code). *Examples:* `npm run build` for frontend, creating a `.jar` file with Maven/Gradle for Spring Boot.

Database Deployment Moving your database from your computer to a cloud-hosted database so the live app can use it. Includes creating the schema, loading starter data, and setting secure credentials. Options: Aiven, Railway, PlanetScale, Supabase, MongoDB Atlas, AWS RDS.

Backend Deployment Hosting your server code on a cloud platform so its APIs are publicly reachable. Options: Render, Railway, AWS, Azure, Heroku-style platforms. You may also use **Docker** to package the app.

Frontend Deployment Hosting your user interface on a platform that serves it to browsers. Options: Vercel, Netlify, GitHub Pages, Firebase Hosting.

Environment Variables Configuration values kept **outside the code** — database URL, passwords, API keys, secret keys, backend URL. They differ between local and production. **Never** upload secrets to GitHub; use `.env` files (added to `.gitignore`) locally and the hosting platform's settings online.

Domain The human-friendly web address (e.g., `myproject.com`) instead of a technical address. It involves a **domain name**, **DNS settings** pointing to your hosting, and **HTTPS/SSL** for secure connections. Free hosting subdomains (like `myapp.vercel.app`) are acceptable for student projects unless told otherwise.

Final Testing Testing the **live** application again after deployment, because things that worked locally can break online (wrong URLs, CORS errors, missing variables). Also called **smoke testing** or **post-deployment testing**.

What to Do for Your Project

1. Create production builds of frontend and backend.
2. Deploy the database and note the connection details securely.
3. Deploy the backend with environment variables configured.
4. Deploy the frontend and point it to the live backend URL.
5. Set up the domain / URL.
6. Test every major feature on the live link.

Deliverables Checklist

- ☐Database live on cloud
- ☐Backend live and reachable
- ☐Frontend live
- ☐Environment variables configured safely
- ☐Live URL working after final testing

---

8. Milestone 6 — Final Submission

Goal: Package, document and present your work professionally.

Terminology

Final Documentation The complete set of documents explaining your project: SRS, design diagrams, API documentation, database schema, user manual (how to use the app), and setup/installation guide.

GitHub Repository The online home of your source code. A good repository has a clean structure, meaningful commit history, a **README.md** (project description, features, tech stack, screenshots, setup steps, live link, team members), a `.gitignore`, and **no secrets**.

Live URL The public link where your deployed project can be opened and used. Make sure it works without needing your computer, and provide demo login credentials if login is required.

PPT (Presentation) A short slide deck to present your project. Suggested flow:

1. Title and team
2. Problem statement
3. Objectives and scope
4. Tech stack and architecture
5. ER diagram and key designs
6. Features with screenshots
7. Testing summary
8. Deployment and live link
9. Challenges faced and learnings
10. Future scope and conclusion

Demo A live (or recorded) walkthrough showing the working project through a real user journey. Practice beforehand, keep a backup recording, and make sure every team member can explain their part.

Project Report A formal written document of the entire project. Typical chapters:

1. Title page, certificate, acknowledgement, abstract
2. Introduction (problem, objectives, scope)
3. Requirement analysis (SRS summary)
4. System design (architecture, ER, DB, API, wireframes)
5. Implementation (modules, code snippets, screenshots)
6. Testing (test cases, results)
7. Deployment details
8. Conclusion and future scope
9. References

Deliverables Checklist

- ☐Final documentation complete
- ☐GitHub repository clean with README
- ☐Live URL submitted
- ☐PPT ready
- ☐Demo prepared (live + backup recording)
- ☐Project report submitted

---

9. Template: Your Project Milestone Plan

Copy this table and fill it in with **your own project's** details.

Project Name: _____

Milestone	Your Project-Specific Tasks	Owner	Start Date	End Date	Status
1. Requirement					
2. Design					
3. Development					
4. Testing					
5. Deployment					
6. Final Submission					

---

10. Worked Example (For Understanding Only)

Project Name: Online Library Management System

Milestone	Example Project-Specific Tasks
1. Requirement	Roles: Admin, Librarian, Student. Use cases: search book, issue, return, pay fine. Non-functional: secure login, fast search
2. Design	React + Spring Boot + MySQL. Entities: User, Book, IssueRecord, Fine. APIs: `/api/books`, `/api/issue`. Wireframes: login, catalog, my books, admin dashboard
3. Development	JWT login, role-based access, book CRUD, issue/return logic, fine calculation, email reminder integration
4. Testing	Test cases for issue/return rules, Postman collection, UI checks on mobile, bug log
5. Deployment	MySQL on cloud, backend on Render, frontend on Vercel, env variables set, live testing

Milestone	Example Project-Specific Tasks
6. Final Submission	README, project report, 10-slide PPT, 5-minute demo, live URL

> ***Remember:*** *This is only an illustration. Your milestones must reflect your project's real features, users and technology.*

---

11. Quick Glossary

Term	Meaning
SRS	Document describing all software requirements
Use Case	A user's interaction with the system to achieve a goal
Functional Requirement	What the system must do
Non-functional Requirement	How well the system must perform (speed, security, usability)
Architecture	Overall structure of components and their communication
ER Diagram	Visual map of entities, attributes and relationships
Primary Key / Foreign Key	Unique row identifier / link between tables
API	Interface through which frontend and backend communicate
Wireframe	Low-detail layout sketch of a screen
Authentication	Verifying who the user is
Authorization	Deciding what the user may access
CRUD	Create, Read, Update, Delete
Frontend / Backend	What users see / server-side logic
Integration	Connecting modules and external services
Unit / API / Integration / UI Testing	Testing code units / endpoints / combined modules / the interface
Regression Testing	Re-testing after fixes to ensure nothing else broke

Term	Meaning
Production Build	Optimized final version of the code
Environment Variables	Configuration and secrets stored outside the code
Domain	The web address of your project
Live URL	Public link of the deployed project
README	Front page of the GitHub repository

---

12. Golden Rules for Students

1. **Plan before you code** — milestones 1 and 2 save time later.
2. **Commit to GitHub regularly** — small, frequent commits.
3. **Build in small pieces** — one feature at a time, test it, then move on.
4. **Never hardcode or upload secrets** — use environment variables.
5. **Document as you go** — don't leave all documentation for the last day.
6. **Every team member must understand the whole project**, not just their own part.
7. **Take mentor reviews at the end of every milestone.**